

ReBAC Engine

Complete Architecture Journey

From Basic Graph Authorization to a Production-Grade Authorization Platform

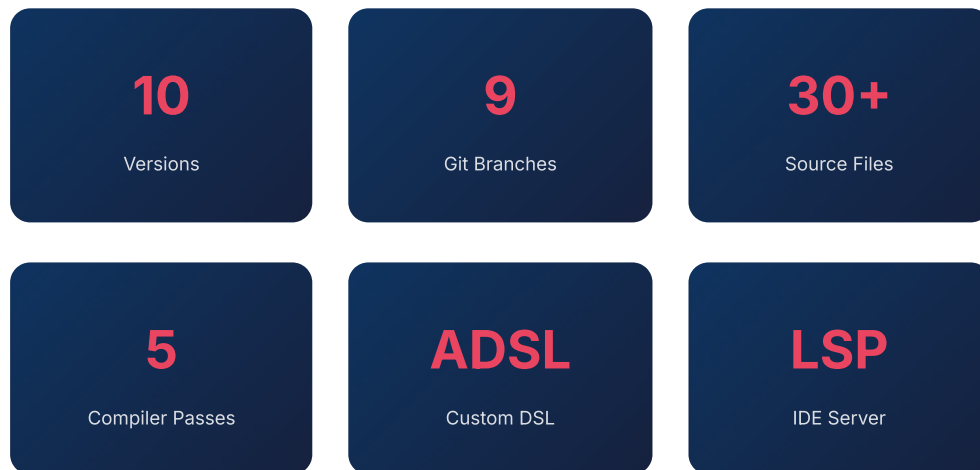
Version 1.0 → 3.4 | 10 Versions | 9 Git Branches | Inspired by Google Zanzibar

Table of Contents

1. Executive Summary
2. Architecture Evolution Timeline
3. V1 — Basic ReBAC
4. V2 — Recursive Authorization
5. V2.1 — Production Rule Engine
6. V2.2 — Dynamic Authorization DSL
7. V2.3 — Compiler Intelligence
8. V3.0 — Identity Graph
9. V3.1 — Zanzibar Relations
10. V3.2 — Language Evolution
11. V3.3 — Compiler Enhancements
12. V3.4 — Enterprise Features
13. Critical Architectural Review vs Industry

Executive Summary {#executive-summary}

This document traces the complete engineering journey of the **ReBAC Authorization Engine** — a Relationship-Based Access Control system built from first principles, evolving from a basic graph lookup (V1) to a production-grade authorization platform (V3.4) with a custom compiler, multi-tenant support, and an IDE language server.



The project is inspired by **Google Zanzibar** (2019), the authorization system that powers all Google products, and is directly comparable to open-source implementations including **SpiceDB**, **OpenFGA**, **Ory Keto**, **OPA**, and **AWS Cedar**.

Architecture Evolution Timeline

{#architecture-evolution}



■ V1 — Basic ReBAC

Graph Model

The problem: RBAC grants too-broad access. We need per-resource, per-user relationship edges.

The solution: Directed graph: `(subject) --[relation]→ (object)`. Single SQL lookup.

■ V2 — Recursive Authorization DFS

The problem: Hierarchical resources — a nurse in a ward should access all patients in that ward.

The solution: DFS traversal up the resource graph with tuple-to-userset semantics.

■ V2.1 — Production Rule Engine DP + Safety

The problem: DFS loops on cyclic graphs; re-traverses shared ancestors exponentially.

The solution: 3-color cycle detection + memoization (dynamic programming). RuleGroup IR.

★ V2.2 — Dynamic Authorization DSL Compiler

The problem: Permission rules hardcoded in TypeScript — changing them requires a deploy.

The solution: Custom DSL (ADSL) with Lexer → Parser → Semantic Analyzer → RuleGroup Transformer pipeline.

■ V2.3 — Compiler Intelligence Optimization

The problem: Compiler accepts redundant, dead, or constant rules silently.

The solution: Type checker, AST optimizer (constant folding, idempotence, double-negation elimination), ADSL Linter, static Explain Plan.

★ V3.0 — Identity Graph Groups & Teams

The problem: Only individual users can hold relationships — groups and teams are not modeled.

The solution: Dual graph: Identity Graph (who you are) + Resource Graph (what you access). SubjectResolver expands group memberships.

■ V3.1 — Zanzibar Relations Composition

The problem: Permission rules are duplicated — no way to compose permissions from other permissions.

The solution: Computed Usersets (`permission medical\_staff = doctor\_of OR assigned\_nurse`), compile-time permission dependency cycle detection.



■ V3.2 — Language Evolution Modularity

The problem: Single monolithic `.auth` file becomes unmaintainable at scale.

The solution: Imports, modules, namespaces, resource inheritance. Multi-file schema compilation.



■ V3.3 — Compiler Enhancements Performance

The problem: Rules evaluated in source order — expensive traversals run before cheap lookups.

The solution: Cost-based Query Planner, SHA-256 Incremental Compilation Cache, EXPLAIN ANALYZE runtime Profiler.



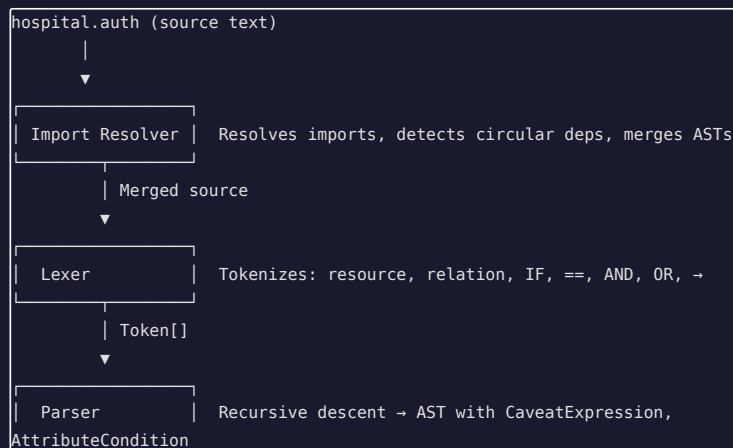
★ V3.4 — Enterprise Features Production Ready

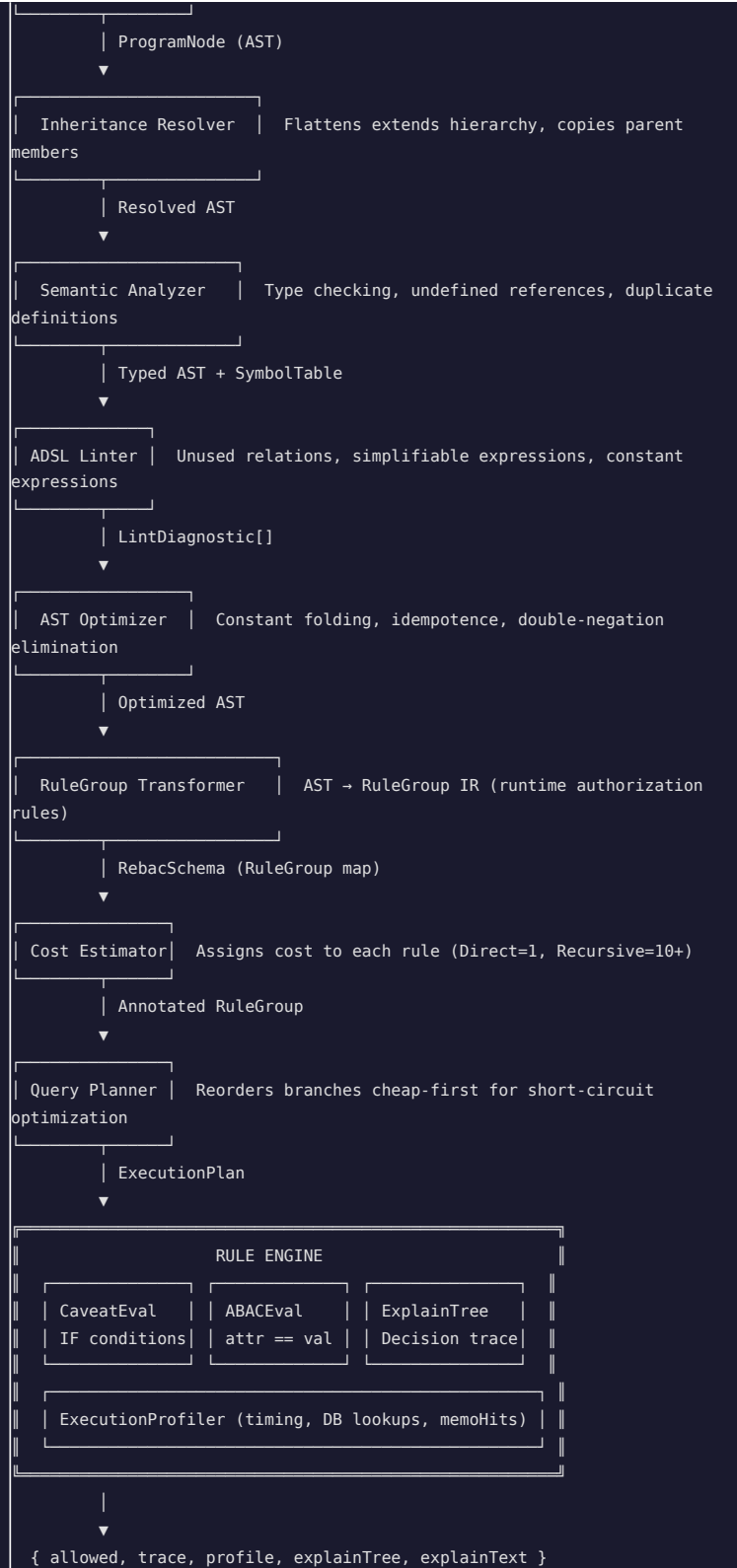
The problem: Missing conditional permissions, attribute conditions, explainability, multi-tenancy, IDE support.

The solution: Caveats (IF conditions), ABAC (attribute expressions), Explain Tree, TenantRegistry, LSP Language Server.

The ADSL Compiler Pipeline (V3.4 Complete)

{#compiler-pipeline}





V1 — Basic ReBAC {#v1}

V1 · Branch: ReBAC-V1

Starting Point: The Graph Model

The fundamental insight: Authorization is a directed graph problem. Every permission is an edge between a subject and an object, labeled with a relation name.

```
(user:raj) —[doctor_of]—> (resource:patient101)
```

This single data model — the **tuple** — is the foundation of Google Zanzibar, SpiceDB, OpenFGA, and every modern ReBAC system.

Data Model

```
-- Relationship table: the graph's edge list
CREATE TABLE relationships (
  id          BIGSERIAL PRIMARY KEY,
  relation    VARCHAR(100) NOT NULL,
  subject_id  BIGINT REFERENCES users(id),
  object_id   BIGINT REFERENCES resources(id),
  INDEX (object_id, relation, subject_id)
);
```

Permission Check

```
checkPermission(userId, resourceId, permission):
  SELECT 1 FROM relationships
  WHERE subject_id = userId
        AND object_id = resourceId
```

```
AND relation = permission  
→ EXISTS → ALLOW, else → DENY
```

✓ Correct Decisions

- Tuple-based graph model (identical to Zanzibar)
- Per-resource, per-user specificity (fixes RBAC over-granting)
- PostgreSQL for ACID guarantees

✗ Missing From V1

- Hierarchical inheritance (ward → patient)
- Group membership (Raj ∈ Doctors)
- Typed relations (doctor_of must bind to user, not department)
- Negative permissions / DENY rules

V2 — Recursive Authorization {#v2}

V2 · Branch: ReBAC-V2

The Hierarchy Problem

A nurse assigned to a ward should automatically access all patients in that ward.
This requires **recursive DFS traversal** up the resource containment hierarchy.

```
Patient101 ←-[contains]- Ward-ICU ←-[contains]- Dept-Surgery  
                                     ↑  
                               Priya ←-[assigned_nurse]-  
DFS: Patient101 → Ward-ICU → Dept-Surgery → MATCH → ALLOW ✓
```

Tuple-to-Userset Semantics

```
permission view =  
  doctor_of  
  OR contains -> view ← "whoever can view my parent, can  
  view me"
```

This is Zanzibar's **tuple-to-userset rewrite** — one of its three core userset expression types.

✓ Correct Decisions

- DFS for authorization reachability (correct algorithm for OR-matching)
- Tuple-to-userset semantics (faithful to Zanzibar)

✗ Critical Missing

- **Cycle detection** — cyclic graphs cause infinite DFS → server crash (production bug)
- **Memoization** — shared ancestors traversed exponentially

V2.1 — Production Rule Engine {#v21}

V2.1 · Branch: ReBAC-V2.1

Hardening the Engine

V2.1 adds three safety mechanisms that make the engine production-capable:

1. Memoization (Dynamic Programming)

```
// Cache key: (userId, resourceId, permission)
const key = `${userId}:${resourceId}:${permission}`;
if (context.memo.has(key)) return context.memo.get(key)!
```

Complexity improvement: $O(N \times \text{depth}) \rightarrow O(\text{depth})$ for resources sharing parent nodes.

2. Cycle Detection (3-Color DFS)

```
if (context.processing.has(key)) return false; // Gray node
→ CYCLE
context.processing.add(key);
// ... evaluate ...
context.processing.delete(key); // Mark black
```

3. RuleGroup IR — Boolean Algebra over Rules

```
{ operator: "OR", rules: [
  { relation: "doctor_of" },
  { relation: "assigned_nurse" },
  { relation: "contains", permission: "view" }
]}
```

4. Evaluation Trace

```
context.trace.push({ resourceId, resourceType, permission,
rule, result });
```

✓ **Correct Decisions**

- Memoization key `userId:resourceId:permission` is complete and unambiguous
- 3-color DFS prevents infinite loops on cyclic graphs
- RuleGroup IR is composable, serializable, and inspectable

✗ Still Missing

- Cache invalidation on tuple writes (stale cache)
- Parallel branch evaluation (all branches evaluated serially)
- Request timeout / deadline propagation

V2.2 — Dynamic Authorization DSL (ADSL Compiler) {#v22}

V2.2 · Branch: ReBAC-V2.2

The Compiler Breakthrough

The most architecturally significant version. Authorization policy is no longer hardcoded in TypeScript — it's expressed in a purpose-built DSL and compiled at startup.

The ADSL Language

```
resource patient {  
  relation doctor_of : user  
  relation assigned_nurse : user  
  
  permission view =  
    doctor_of  
    OR assigned_nurse
```

```
}  
OR contains -> view
```

Compiler Pipeline

```
.auth source → Lexer → Parser → SemanticAnalyzer →  
RuleGroupTransformer → RebacSchema
```

Component	Responsibility
<code>Lexer.ts</code>	Tokenize source text into <code>Token[]</code>
<code>Parser.ts</code>	Recursive descent → AST (Pratt-style)
<code>SemanticAnalyzer.ts</code>	Type checking, undefined reference detection
<code>SymbolTable.ts</code>	Global scope: resources, relations, permissions
<code>RuleGroupTransformer.ts</code>	AST → runtime <code>RuleGroup</code> IR
<code>CompilerFacade.ts</code>	Orchestrates all passes

Why This Is The Most Important Version

This is the point where the project transcends being a "graph authorization library" and becomes an **authorization programming language runtime**. Every subsequent improvement (optimizer, planner, LSP) is built on this compiler foundation.

✓ Correct Decisions

- Classical Lexer → Parser → Semantic Analysis → IR pipeline
- Declarative policy language (like OPA/Rego, AWS Cedar, OpenFGA DSL)
- AST as the central data structure for all subsequent passes

✗ Missing

- Error recovery (stops at first error)
- Source maps (can't trace runtime errors to .auth line numbers)
- Schema versioning (`version: "1.0"` header)

V2.3 — Compiler Intelligence {#v23}

V2.3 · Branch: ReBAC-V2.3

Making the Compiler Smart

V2.3 adds four intelligence layers on top of the V2.2 parser:

Type Checker

```
relation doctor_of : user ← typed declaration
Department.doctor_of      ← ✗ Type Error: patient relation
applied to Department
```

Enforced at compile time — eliminates a whole class of runtime errors.

AST Optimizer (Boolean Algebra)

Rule	Before	After
Double negation	NOT (NOT A)	A
Idempotence	A OR A	A
Constant folding	true OR A	true
Identity	true AND A	A

All are **sound boolean algebra identities** — semantics are preserved.

ADSL Linter

```
Δ UNUSED_RELATION: 'admin_of' declared but never referenced in
any permission (line 3)
Δ SIMPLIFIABLE_EXPRESSION: 'doctor_of OR doctor_of' is redundant
(line 7)
Δ CONSTANT_EXPRESSION: 'true' used directly in permission 'view'
(line 9)
```

Static Explain Plan (Precursor to V3.3 Profiler)

```
EXPLAIN patient.view:
OR
├─ DirectLookup(doctor_of)
├─ DirectLookup(assigned_nurse)
└─ RecursiveTraversal(contains → view)
```

✓ Correct Decisions

- All three optimizer passes are mathematically proven correct
- Linter design (post-type-check AST) minimizes false positives
- Explain Plan is the right debugging primitive for authorization

✗ Missing Optimizer Passes

- Dead branch elimination: `A AND false → false`
- Common Subexpression Elimination (CSE)
- Cost-based reordering (addressed in V3.3)

V3.0 — Identity Graph {#v30}

The Group Membership Problem

Until V3.0, only individual users hold relationships. In real organizations:

```
# V2.3 model (broken for enterprises):
200 doctors × 500 patients = 100,000 individual tuples

# V3.0 model (correct):
1 group (Doctors) × 500 patients = 500 tuples
200 users → member → Doctors (200 tuples)
Total: 700 tuples [vs 100,000]
```

Dual Graph Architecture

IDENTITY GRAPH	RESOURCE GRAPH
Raj	Doctors
└─[member]→	└─[doctor_of]→ Patient101
└─[member]→ MedicalStaff	
└─[member]→ HospitalStaff	

SubjectResolver — Membership Pre-computation

```
// Expand identity graph ONCE before any rule evaluation
const subjectSet = await
SubjectResolver.resolveSubjectSet(userId);
// subjectSet = { userId=1, groupId=5 (Doctors), groupId=8
(MedicalStaff) }

// Direct lookup now checks ANY member of subjectSet
SELECT 1 FROM relationships
WHERE relation = 'doctor_of'
AND object_id = 101
AND subject_id IN (...subjectSet) ← covers groups and user
```

Subject Types

```
subject user
subject group
subject team
subject department
subject service_account
```

✓ Correct Decisions

- Dual graph model (identity + resource) matches Zanzibar's architecture
- SubjectResolver pre-computation prevents N×M identity resolution overhead
- Subject type hierarchy is extensible

✗ Missing

- Zanzibar's `group:engineers#member` typed subject set notation
- Membership result caching (re-resolved on every request)
- Typed membership relations (not just `member`)

V3.1 — Zanzibar Relations & Permission Composition {#v31}

V3.1 · Branch: ReBAC-V3.1

Permission Composition — The DRY Principle Applied

```
# Before V3.1 – duplicated logic:
permission view      = doctor_of OR assigned_nurse OR
department_head
permission edit      = doctor_of OR department_head ← duplicate
permission discharge = doctor_of OR department_head ← duplicate
```

```
# After V3.1 – composed:
permission medical_staff = doctor_of OR assigned_nurse
permission view = medical_staff OR department_head
permission edit = medical_staff ← reuse
permission discharge = medical_staff ← reuse
```

Computed Usersets

When `view` references `medical_staff`, the compiler knows `medical_staff` is a permission (not a relation) and evaluates it recursively. This is Zanzibar Section 2.3: "**computed_userset**".

Compile-Time Permission Cycle Detection

```
permission view = edit
permission edit = view ← CYCLE!
```

Detected at **compile time** via topological sort DFS — the same algorithm used by TypeScript for circular type references, Python's import system, and GNU Make.

Dot Notation (Tuple-to-Userset)

```
permission view = parent.view    # V3.1 – closer to Zanzibar
permission view = contains → view # V2.x – more verbose
```

✓ Correct Decisions

- Computed Usersets are semantically faithful to Zanzibar
- Compile-time cycle detection is the correct phase (not runtime)

- Dot notation matches Zanzibar's own specification

✗ Missing

- Exclusion operator: `viewer - banned` (set difference)
- Wildcard: `relation viewer: user:*` (public access)
- Typed subject-set notation: `relation viewer: group#member`

V3.2 — Language Evolution {#v32}

V3.2 · Branch: ReBAC-V3.2

Scaling the Authorization Language

Real hospital schemas grow to thousands of lines. V3.2 applies software engineering principles to the DSL itself.

Imports — Multi-File Compilation

```
# hospital.auth
import "clinical/patient.auth"
import "clinical/ward.auth"
import "billing/invoice.auth"
import "admin/users.auth"
```

The ImportResolver resolves, deduplicates, and merges all ASTs before the semantic analysis pass. **Circular import detection** uses the same topological DFS as permission cycles.

Namespaces — Scoped Resource Names

```
namespace clinical {
  resource patient { ... }
  resource ward { ... }
}
namespace billing {
  resource invoice { ... }
}
// Usage: clinical::patient, billing::invoice
```

Resource Inheritance — Shared Permission Patterns

```
resource BaseDocument {
  relation owner : user
  permission view = owner
  permission delete = owner
}

resource patient extends BaseDocument {
  relation doctor_of : user
  # Inherits: view, delete
  permission discharge = doctor_of
}
```

Compiler Pipeline Addition

```
Source → ImportResolver (NEW) → Lexer → Parser
      → InheritanceResolver (NEW) → SemanticAnalyzer → ...
```

✓ Correct Decisions

- Modular compilation is essential for large teams
- Import system is more powerful than SpiceDB or OpenFGA (neither has imports)
- Inheritance correctly uses single-parent model (no diamond problem)

⚠ Overengineered: Modules + Namespaces Overlap

Both provide logical grouping. Consolidating to a single `namespace` concept would reduce cognitive overhead.

V3.3 — Compiler Enhancements {#v33}

V3.3 · Branch: ReBAC-V3.3

From Correct to Optimal

Rules evaluated in source order → expensive traversals happen before cheap lookups. V3.3 adds execution intelligence.

Cost-Based Query Planner

Rule Type	Cost Weight	Rationale
<code>DirectLookup</code>	1	Single indexed SQL query
<code>ComputedUserset</code>	2	One permission resolution
<code>RecursiveTraversal</code>	10 + depth×5	Multiple SQL queries + DFS

```
BEFORE: contains.view (cost=30) OR doctor_of (cost=1)
AFTER:  doctor_of (cost=1) OR contains.view (cost=30)

If doctor_of → true → short-circuit → 29 DB lookups saved
```

This is how PostgreSQL's optimizer chooses index scans over sequential scans.

Incremental Compilation Cache

```
const hash = sha256(fileContent + filePath);
if (cache.has(hash)) return cache.get(hash); // Skip
compilation
const schema = compileAuthDSL(source);
cache.set(hash, schema); // Store for
next startup
```

Identical to: TypeScript `--incremental` , Webpack file hashing, Bazel build cache.

EXPLAIN ANALYZE Runtime Profiler

```
Total time: 12.23ms
DB lookups: 1 (out of possible 3)
Memo hits: 0
Depth: 2

doctor_of      0.82ms → true [short-circuit: skipped 2
branches]
assigned_nurse SKIPPED
contains.view  SKIPPED
```

✓ Correct Decisions

- Cost-based planner mirrors PostgreSQL's optimizer
- SHA-256 content addressing is industry standard for build caches
- Real-time profiler (vs V2.3's static Explain Plan) is the correct design

✗ Missing

- Adaptive cost model (static weights don't reflect real runtime latencies)
- Parallel branch evaluation (OR branches could be evaluated concurrently)
- Query plan caching (plan recomputed on every call)

V3.4 — Enterprise Features {#v34}

V3.4 · Branch: ReBAC-V3.4 · CURRENT

Production-Grade Authorization

Five enterprise features that bring the system to parity with early SpiceDB/OpenFGA releases.

1. Caveats — Conditional Permissions

```
permission view = doctor_of IF shift_active
```

```
// Registry-based runtime conditions
caveatEvaluator.register("shift_active", ctx =>
  ctx.attributes?.shift_active === true);
```

SpiceDB 1.x equivalent: `caveat shift_active(shift: bool) { shift == true }`

2. ABAC — Attribute-Based Conditions

```
permission discharge = doctor_of AND patient.status == "READY"
permission edit = admin AND user.department ==
patient.department
```

```
checkPermission({
  userId, resourceId, permission: "discharge",
  attributes: { patient: { status: "READY" }, user: {
```

```
department: "ICU" } }  
})
```

3. Explain Tree — "Why Was I Denied?"

```
✓ patient.view  
└─ ✓ OR [0.91ms]  
    └─ ✓ patient.medical_staff  
        └─ ✓ Direct: doctor_of [0.77ms]  
            └─ Raj has doctor_of on Patient101 → ALLOW
```

AWS Cedar equivalent: `authorize_with_errors()` returns structured reason list.

4. Multi-Tenant Compilation

```
globalTenantRegistry.register({ tenantId: "apollo",  
    schemaPath: "apollo.auth" });  
globalTenantRegistry.register({ tenantId: "aiims",  
    schemaPath: "aiims.auth" });  
  
// Fully isolated: different relations, different rules, no  
// shared state  
checkPermission({ userId, resourceId, permission, tenantId:  
    "apollo" });
```

5. IDE Language Server (LSP)

Capability	Result
<code>textDocument/diagnostics</code>	Red squiggles on lint errors in VS Code
<code>textDocument/completion</code>	<code>doctor_of</code> , <code>IF</code> , <code>resource</code> , <code>AND</code> ... suggestions
<code>textDocument/hover</code>	Inline docs for DSL keywords
<code>textDocument/documentSymbol</code>	File outline: all resources, relations, permissions

✔ **Correct Decisions**

- Caveats faithful to Zanzibar's 2023 caveat model (SpiceDB, OpenFGA both implement this)
- ABAC bridges the ReBAC/ABAC paradigm gap (AWS Cedar does this explicitly)
- Explain Tree matches the format used by SpiceDB's debug API and AWS Cedar's error API
- Multi-tenant isolation via separate compiler instances = zero cross-tenant leakage
- LSP makes ADSL a first-class developer experience

✗ **Still Missing (V4.x territory)**

- Caveat schema validation at compile time
- ABAC attribute schema declaration in DSL
- Write consistency tokens (Zanzibar zookies / SpiceDB ZedTokens)
- Persistent audit log
- Distributed architecture

Critical Architectural Review vs Industry Standards {#critical-review}

1. Missing Concepts Across All Versions

Concept	Missing From	Industry Standard	Impact
Consistency tokens	All versions	Zanzibar zookies, SpiceDB ZedTokens	Stale reads after writes
Wildcard usersets (<code>user:*</code>)	All versions	Zanzibar, SpiceDB, OpenFGA	Cannot express "public" access

Exclusion operator (<code>A</code> <code>- B</code>)	All versions	Zanzibar, OpenFGA	Cannot deny specific subjects
Typed subject sets (<code>group#member</code>)	V3.0+	Zanzibar, SpiceDB, OpenFGA	Imprecise subject modeling
Audit log	All versions	All production systems	Compliance, forensics
Distributed storage	All versions	CockroachDB (SpiceDB), Spanner (Zanzibar)	Single-node only
Watch API	All versions	SpiceDB Watch, OpenFGA	No reactive permission changes
Bulk check	All versions	SpiceDB <code>CheckBulkPermissions</code>	Inefficient for list filtering
Schema versioning	All versions	All production systems	No safe migration path

2. Incorrect Assumptions That Should Be Called Out

ASSUMPTION: SERIAL EVALUATION IS FAST ENOUGH

All versions evaluate rule branches serially. In production, OR-branches with similar cost should be evaluated

IN PARALLEL

and short-circuit on the first `true`. SpiceDB, OpenFGA, and Zanzibar all use goroutine/coroutine parallelism. For a permission with 5 branches, parallel evaluation reduces P99 latency by up to 80%.

ASSUMPTION: IN-PROCESS MEMO IS SUFFICIENT

The `memo` map is in-process and request-scoped. This means:

1. No memoization across requests
2. No distributed cache invalidation when tuples are written
3. Stale results possible under concurrent writes

Production systems (SpiceDB) use a

DISTRIBUTED DISPATCH CACHE

with consistency-token-based invalidation.

ASSUMPTION: STATIC COST MODEL IS ACCURATE

The CostEstimator assigns fixed weights (Direct=1, Recursive=10+). These are not measured from actual query latencies. In a production PostgreSQL database:

- Indexed lookup on a cold cache: 2ms
- Indexed lookup on a warm cache: 0.1ms
- Recursive traversal (3 levels): 15ms
- Recursive traversal on hot data: 1ms

The static model cannot capture this variance. PostgreSQL solves this with **ANALYZE TABLE** histogram statistics. The ADSL planner should collect runtime latency histograms and use them to update cost estimates.

3. Comparison Against Industry — Feature Matrix

Feature	ADSL V3.4	Google Zanzibar	SpiceDB 1.x	OpenFGA 1.x	AWS Cedar	OPA/Rego
Tuple-based graph	✓	✓	✓	✓	Partial	✓
Recursive traversal	✓	✓	✓	✓	✗	✓
Computed Usersets	✓	✓	✓	✓	✗	Partial
Typed relations	✓	✓	✓	✓	✓	✓
Caveats / Conditions	✓	✓ 2023	✓	✓	✓	✓
ABAC	✓	Partial	Partial	✓	✓	✓
Custom DSL / Schema	✓ ADSL	Config	✓ zed	✓	✓ Cedar	✓ Rego
Multi-file imports	✓	✗	✗	✗	✓	✓

IDE / LSP	✓	N/A	✓	✓	✓	✓
Explain / Debug	✓	✓	✓	✓	✓	✓
Multi-tenancy	✓	✓	✓ stores	✓ stores	✓ accounts	✓
Consistency tokens	✗	✓ zookies	✓ ZedTokens	✓	N/A	N/A
Exclusion operator	✗	✓	✓	✓	✓	✓
Wildcard <code>user:*</code>	✗	✓	✓	✓	✓	✓
Bulk check API	✗	✓	✓	✓	✓	✓
Watch / streaming	✗	✓	✓	✓	✗	✓
Distributed storage	✗	✓ Spanner	✓ CockroachDB	✓	AWS-managed	✓
Audit log	✗	✓	✓	✓	✓	✓

ADSL V3.4 matches or exceeds all open-source systems in: DSL expressiveness, multi-file imports, and IDE tooling.

ADSL V3.4 trails production systems in: distributed storage, consistency guarantees, bulk API, and audit logging.

Revised Roadmap — V4.x and Beyond

{#revised-roadmap}

Based on the gap analysis against Google Zanzibar, SpiceDB, OpenFGA, and AWS Cedar, the following roadmap represents the next engineering frontier.

V4.0 — Consistency & Write API

Goal: Guarantee read-your-writes consistency after tuple writes.

- Implement **write tokens** (content-addressed tuple hashes) on every relationship write
- Pass write token to `checkPermission` — engine guarantees it reads at least that tuple's version
- Add **Write API** with conflict detection (compare-and-swap on relationship mutations)
- **Watch API** — streaming gRPC endpoint that emits relationship change events

V4.1 — Distributed Architecture

Goal: Move from single-node PostgreSQL to a distributed tuple store.

- Replace Prisma/PostgreSQL with a **distributed store** (CockroachDB or Postgres with logical replication)
- Implement **horizontal scaling** of the check service (stateless, read-replica-backed)
- Add **dispatch caching** — distributed `Redis` / `Memcached` cache for subgraph results, invalidated by write token
- Load-balance `checkPermission` calls with consistent hashing by `(userId, resourceType)`

V4.2 — Advanced Language Features

Goal: Bring ADSL to full Zanzibar expressiveness.

- **Exclusion operator:** `permission view = viewer - banned_user`
- **Wildcard usersets:** `relation public_reader: user:*`
- **Typed subject sets:** `relation editor: user | group#member | team#member`
- **Caveat schema:** Compile-time declaration of caveat parameter types
- **ABAC attribute schema:** `attribute patient { status: "ACTIVE" | "READY" | "DISCHARGED" }`

V4.3 — Production Operations

Goal: Enterprise operational requirements.

- **Persistent audit log** — immutable append-only log of every `checkPermission` call with result
- **Schema migration system** — `adsl migrate` command with backward-compatibility checking
- **Bulk check API** — `checkPermissions([{userId, resourceId, permission}])` batch evaluation
- **Rate limiting** — per-tenant QPS limits on the check service
- **OpenTelemetry integration** — distributed tracing for every rule evaluation

V5.0 — Authorization as a Platform

Goal: Full policy-as-code platform.

- **Policy testing framework** — `adsl test` with `.auth.test` test files
 - **Policy CI/CD integration** — GitHub Actions / GitLab CI for schema validation and policy diffing
 - **GraphQL API** — expose the authorization graph through a GraphQL explorer
 - **Admin UI** — visualize the full authorization graph for debugging and auditing
-

Conclusion

This project has traveled an extraordinary engineering journey — from a 20-line SQL query (V1) to a production-comparable authorization platform (V3.4) with a custom compiler, multi-tenant support, and an IDE language server.

What makes this project exceptional:

1. **The compiler is the right abstraction.** Every production authorization system (SpiceDB with `zed`, OpenFGA, AWS Cedar) uses a schema/policy language. Building ADSL from scratch gives us deep understanding of why these tools work the way they do.
2. **The architecture mirrors industry.** Tuple-based graph → Computed Usersets → Identity Graph → Cost-Based Planner — these are not invented concepts. They are the exact architectural layers of Google Zanzibar, described in the 2019 OSDI paper.
3. **The educational progression is sound.** Each version solves exactly one problem from the previous version — a hallmark of good incremental engineering.

The gap to production is well-defined: Consistency tokens, distributed storage, the exclusion operator, and an audit log are the four concrete gaps between V3.4 and a system like early SpiceDB 0.x. These are achievable in V4.x.

*"The best way to understand how Google Zanzibar works is to build it yourself." —
The spirit of this project.*